

第四题 蚂蚁寻路问题

【问题描述】

小蚂蚁是蚂蚁王国的一名快递员，每天需要从王国的仓库运送食物。在一个由 n 行 m 列组成的网格中，一只小蚂蚁要从网格的左下角出发，爬到网格的右上角。小蚂蚁每次只能向上或向右移动一格，不能向下或向左移动。

小蚂蚁想知道，从起点 $(0,0)$ 到终点 (n,m) 一共有多少条不同的路径。由于答案可能非常巨大，我们只需要知道答案的最后 100 位数字。

【输入描述】

输入只有一行，包含两个整数 n 和 m ，分别表示网格的行数和列数。

【输出描述】

输出共 10 行，每行十个数字，没空格间隔，表示答案的最后 100 位。如果答案不足 100 位，则在前面用 0 补足。

【输入样例 1】

2 2

【输出样例 1】

0000000000
0000000000
0000000000
0000000000
0000000000
0000000000
0000000000
0000000000
0000000000
0000000006

【数据范围】

对于 30% 的数据， $0 < n, m \leq 10$

对于 50% 的数据， $0 < n, m \leq 100$

对于 100% 的数据， $0 < n, m \leq 50000$

【示例解释】

2x2 的网格中，从左下角到右上角的路径数为 6，具体路径如下：

右→右→上→上
右→上→右→上
右→上→上→右
上→右→右→上
上→右→上→右
上→上→右→右

题目解析

题目分析：

本题题意为给定 n 和 m ，求从 $(0, 0)$ 开始，每次只能向上或者向右走一步，走到 (n, m) 的路线方案数。本题考察高精度乘法，素数筛法，组合数学等知识。

解题思路：

想要求解从 $(0,0)$ 到达 (n, m) 的路线数，可以使用标数法计算。可以设 $f(i, j)$ 为从 $(0,0)$ 到达 (i, j) 的路线数量，由于 (i, j) 只能从 $(i-1, j)$ 和 $(i, j-1)$ 走过来，因此可以得到 $f(i, j) = f(i-1, j) + f(i, j-1)$ ，初始情况 $f(0, 0) = 1$ 。这种做法时间复杂度为 $O(nm)$ ，可以通过 30% 的数据。由于答案要求输出后 100 位，也就是说答案会很大，所以需要使用高精度进行计算。进行计算的时候需要注意，由于数字本身很大，但是答案只需要输出后 100 位，所以只需要存储后 100 位，对后 100 位做加法即可。这种情况下时间复杂度为 $O(100*n*m)$ ，可以通过 50% 的数据。

对于满分做法，可以考虑数学的做法。可以发现无论如何走，走的总步数一定是走 $n+m$ 步，其中有 n 步是向右走， m 步是向上走。所以路线的数量是从总共的 $n+m$ 步中挑选出 n 步向右，剩余的 m 步向上的方案数，也就是从 $n+m$ 个物品中选出 n 个物品的方案数，为 $C(n+m, n)$ 。

计算组合数的方式有两种，一种是通过递推式计算，但是 100% 的数据下使用递推式计算会超时，因此只能使用第二种方式使用阶乘的公式计算， $C(n+m, n) = (n+m)! / (m! * n!)$ 。由于结果很大，这里需要使用高精度进行计算。当然需要注意的是，高精度只需要计算后 100 位即可。

在计算的时候由于式子中还有除法，可以使用质因数分解约分的形式去掉式子中的除法。约分的时候只需要计算出每个质数 p 在 $n!$ 中的指数大小即可，用指数的减法代替除法操作。因此需要先使用素数筛法生成素数表，然后再枚举质数，计算出该数在 $n+m!$, $n!$, $m!$ 中的指数大小即可。

其中 $n!$ 中 p 的指数为 $n/p + n/p^2 + \dots + n/p^k$ 其中 $p^k \leq n < p^{k+1}$

因此需要实现这些函数：求解质数表的素数筛法、计算阶乘中 p 的质数个数函数、计算高精度乘低精度乘法函数（只需计算后 100 位）。

计算出结果后，再按照 **10 * 10** 的格式进行输出。

参考代码：

```
#include <iostream>
#include <vector>
#include <cstring>
#include <algorithm>
using namespace std;

const int MAX_N = 100000; // m+n 的最大值
const int DIGITS = 100; // 需要保留的位数

// 高精度乘法（只保留后 100 位）
void multiply(vector<int> &result, int num) {
    int carry = 0;
    for (int i = 0; i < DIGITS; i++) {
        long long temp = (long long)result[i] * num + carry;
        result[i] = temp % 10;
        carry = temp / 10;
    }
    // 超过 100 位的进位直接丢弃
}

// 计算 n! 中质数 p 的指数
int calcExponent(int n, int p) {
    int count = 0;
    int temp = n;
    while (temp > 0) {
        temp /= p;
        count += temp;
    }
    return count;
}

// 埃拉托斯特尼筛法生成质数
vector<int> generatePrimes(int limit) {
    vector<bool> isPrime(limit + 1, true);
    vector<int> primes;

    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i <= limit; i++) {
        if (isPrime[i]) {
            primes.push_back(i);
        }
    }
}
```

```

        for (long long j = (long long)i * i; j <= limit; j += i) {
            isPrime[j] = false;
        }
    }
}

return primes;
}

int main() {
    int m, n;
    cin >> m >> n;

    int total = m + n;

    // 生成所有需要的质数
    vector<int> primes = generatePrimes(total);

    // 初始化结果数组，只保留 100 位
    vector<int> result(DIGITS, 0);
    result[0] = 1; // 初始化为 1

    // 计算组合数  $C(\text{total}, m) = \text{total}! / (m! * n!)$ 
    // 使用质因数分解的方法
    for (int p : primes) {
        if (p > total) break;

        // 计算质数 p 在组合数中的指数
        int exponent = calcExponent(total, p) - calcExponent(m, p) -
calcExponent(n, p);

        // 将  $p^{\text{exponent}}$  乘到结果中
        for (int i = 0; i < exponent; i++) {
            multiply(result, p);
        }
    }

    // 输出结果（按题目要求的格式）
    for (int i = 9; i >= 0; i--) {
        for (int j = 9; j >= 0; j--) {
            cout << result[i * 10 + j];
        }
        cout << endl;
    }
}

```

```
    return 0;  
}
```